

LAB 3 – ARCHITECTURE SELECTION AND JUSTIFICATION

Name: Amrutha Kattimani

SRN: PES1UG24CS054

[Project: Local Sports Tournament & Bracket Manager]

Architecture Selection

We chose the Microservices Architecture for the Local Sports Tournament & Bracket Manager.

Architectural Choice

The system is divided into independent services such as Team Management Service, Tournament and Bracket Service, Match and Score Service, Tournament Monitoring Service, and Notification Service. An API Gateway provides a common entry point for the Tournament Management UI.

Reasons for Choosing Microservices

1. Independent Scalability:

Different tournament functions may experience different workloads. For example, Match and Score Service may receive frequent requests during live matches, while Team Management may have comparatively fewer requests. Microservices allow individual services to be scaled independently according to their workload.

2. Fault Isolation and Independent Deployment:

A failure in one service should not bring down the entire tournament system. For example, if the Notification Service becomes unavailable, team management and match recording can continue. Individual services can also be updated or deployed without changing the complete system.

Security Advantage

The API Gateway provides a controlled entry point to the system. Requests can be authenticated and authorized at the gateway before being forwarded to the required service. Service-level access control can further restrict which operations each component is allowed to perform.

Performance Benefit

Microservices allow frequently used services, such as the Match and Score Service, to be scaled independently. This helps the system handle a large number of match-score updates concurrently without unnecessarily scaling other services.

Conclusion

Microservices are suitable for the Local Sports Tournament & Bracket Manager because they provide modularity, independent scalability, fault isolation, and controlled communication between system components. The architecture supports the system's tournament management, live score handling, monitoring, and notification requirements while allowing individual services to evolve independently.